SE 3170: Introduction to Software Testing

Lab 6

Testing Distributed Systems: The ATM Machine and the Utility Company

15 points

Goal

Acquire an ability to develop and conduct appropriate experimentation, analyze and interpret data, and use engineering judgment to draw conclusions

Objectives

The primary objective of Lab 6 is to build a distributed system in which three subsystems—a pair of Bank systems (handling Checking and Savings accounts) and a Utility Company system—interact to support common financial actions for users. Students will create a command-line application that allows users to deposit, withdraw, transfer funds, pay bills, and check balances, while ensuring that all operations and data are persistently stored. Effective implementation includes thorough testing of both individual code logic and the user interface, echoing the expectation to provide clear documentation and thorough records, much like previous labs.

Constraints

Strict account transaction limits are enforced. Checking accounts allow up to $ 5,000 in deposits and $500 in withdrawals daily. Savings accounts also allow up to $ 5,000 in deposits but only $100 in daily transfers to Checking, and do not authorize direct withdrawals or bill payments. Accounts must never have a negative balance (no overdraft), and each user is limited to one Checking and one Savings account.

Utility Company accounts require unique credentials and are assigned a six-digit account number at creation. The lab must be completed using the command line only, with clear commenting, robust exception handling, and all account/transaction data stored persistently. Testing is required for all features, with attention to both valid and invalid use cases as well as edge conditions. As with earlier assignments, each submission must be original and individually submitted.

Java Project Structure

Create a single project "Lab6_<YourName>" with ***src*** and ***test*** directories. To make the workflow more convenient, it is recommended to use separate packages for each distinct portion.

Use clear class names, mirroring the previous lab's structure for consistency and grading. Ensure all test code and outputs are captured in a PDF report, and the project is exported as an archive for submission, as you do in previous assignments.

➢ Section 1 – Project Setup:

- **Designing and conducting the experiment:** In this assignment, you will design, build and test a distributed system of multiple subsystems, a bank system made of a checking and a savings account, and an external utility company system.
- This Lab Assignment can be done individually or in a team of two (max).
- If two students want to collaborate on this assignment, each one will still submit individually
- In your submission, you must indicate if you did the assignment alone or with another person (both names and IDs must be included).

You will start with an ATM machine. The machine will allow users access to their bank accounts and manage their money transfers and bill payments. The bank has the following rules:

1- Checking Account:
   - The user can have one checking account.
   - The user can deposit up to $ 5,000 per day into this account using one or more transactions.
   - The user can withdraw a maximum of $500 per day using one or more transactions.
   - The user can transfer any amount of money from their checking account to their savings account.
   - The user can use this checking account to pay bills to their utility company.
   - The user can check the balance of their checking account.
   - The bank does not allow overdraft, the balance can NOT be negative.

2- Saving Account:
   - The user can have one saving account.
   - The user can deposit up to $5000 per day using one or more transactions.
   - The user can NOT withdraw any money from this account.
   - The user can transfer up to $100 per day from their savings account to their checking account using one or more transactions.
   - The user can NOT use this account to pay bills.
   - The user can check the balance of their savings account.
   - The bank does not allow overdraft, so the balance can NOT be negative.

3- The Utility Company
   - The user will create a new account at the utility company using a username and a password.
   - The utility company will give the user a new account number (6-digit) generated automatically.
   - The user has access to their utility company account using their username (or utility account number) and a password.
   - The user can log in to their utility account at any time and view their bill payment history, including the last three paid bills.
   - The user can also check the next bill payment amount and due date.

All three accounts are persistent.

➢ Section 2 – In-Class Activity: System Design Deliverable (3 points)

Submit a document showing how you will build the system. Your submission will include the following:

1- A schematic diagram of the three subsystems (Bank Checking, Bank Saving, and Utility)
2- UML Class diagram showing the classes names and functionality as well as the methods in each class. You can use UML diagrams or pseudo-code
3- Initial code obtained by ChatGPT.

- **Section 2 submission is due at the end of the class.**

  - ✓ **Ask ChatGPT to give system design details, including components, description, code, and diagrams.**
  - ✓ **Repeat and refine your prompts to ChatGPT and submit all prompts and responses."**

➢ Section 3 – Testing and System Deliverables (12 points)

Submit a document that includes the following:

1- **Source code**:
   a. Use exception handling and add appropriate comments to your code.
   b. Use appropriate persistent storage structures for the bank and the utility company accounts.

c. Interaction I/O will be done using the command line.
d. Add a Read-Me file of how to deploy and use your assignment.
e. Use your own software code components; do not use any existing application.

2- **Screenshots** showing the steps of user actions and the corresponding output.

3- **Code Testing**:
   a. A comprehensive test plan of your software, including:
      i. Test designs and execution of your project functions and algorithms, storage structures, and user actions for handling the functionalities of their two bank accounts and the utility company (valid and invalid transactions shall be covered in your tests)
      ii. Regarding data storage:
         1. Test for null storage
         2. Test for null element with multiple elements
         3. Test for a null single element
         4. Test for incompatible types
         5. Test for empty elements, and two normal cases (single and multiple elements).
      iii. Test outcomes (screenshots and description)
      iv. Sufficient transactions for testing

4- **UI Testing**:

To meet the requirements outlined above, **design an experiment to evaluate the UI**.

i. Use two testing methods:
   1. Functionality
   2. Boundary.
ii. Provide one test criterion for each method.
iii. Design a suitable Test Requirement (TR) set for each criterion.
iv. Provide all test cases for your TR set.
v. Run your test cases
vi. **Collect the results** of your UI tests using the following **table**:
   1. Test method
   2. Test criterion
   3. Test input values
   4. Test expected output
   5. Test actual output
   6. Test success/fail.
vii. **Provide your own analysis and interpretation** of the results of your UI tests
viii. **Draw your own conclusion** (2-lines) of how easy the UI would be from a user perspective
ix. Take **screenshots** of your test results
x. Write an analysis of your results and identify any unexpected behavior or wrong results.
xi. Add all lab requirements to the **report**.

Notes:
There is some overlap between code testing and UI testing, but they are performed from different perspectives. You can reuse some tests from 3 in 4 if it applies.

➢ Deliverables:

- ✓ **PDF** Report file(s): ***"Lab-6.pdf"***
  - ✓ Make sure your submission has a proper layout with clear titles and comments.
  - ✓ Organize and insert your screenshots of testing, test results, and other requirements into a PDF file.

- ✓ Include all material (code, test cases, UML, etc.) obtained by ChatGPT, clearly indicating that they are AI-Generated.
- ✓ Include all **verified** AI-material you used in your project, clearly indicating that they are AI-Generated AND verified by you if applicable. Highlight the changes you made in your verification.
- ✓ Include all material genuinely created by you, and indicate that they are not AI-generted.
- ✓ Add your answers to any questions.

✓ **Archived** file: *"**Lab-6.zip**"*

- ✓ Right-click on the project name from the package/project explorer panel -> export -> General -> Archive File.

✓ Submit files SEPARATELY on Canvas.